
CASE STUDY / 2026

Sports Catalog Scraper

Targeted Python crawler for e-commerce product data and asset extraction.

Python • requests • BeautifulSoup4 — Web scraping & data extraction

PREPARED BY

Mudasir Shafiq

Full-stack & data engineer · chmudasir.fr@gmail.com

CONTENTS

Table of Contents

01	Project Overview	Page 3
02	Crawl Pipeline	Page 4
03	Field Extraction Map	Page 5
04	Technology Stack	Page 6
05	The Challenge	Page 7
06	The Approach & Filter Funnel	Page 8
07	Code Spotlight	Page 9
08	Outcome & Engagement	Page 10

This document presents an in-depth look at the Sports Catalog Scraper — what it does, the crawl pipeline, how HTML structure maps to a clean CSV, and the technical decisions behind it. Every diagram, snippet, and stat is grounded directly in the project source.

SECTION 01

Project Overview

What the scraper does, what it produces, and why it stays small on purpose.

The **Sports Catalog Scraper** is a focused Python crawler that turns a public e-commerce storefront — a sports-uniform retailer at *fcsuniforms.com* — into a structured CSV plus a folder of downloaded product images, narrowed down to the categories that matter to the buyer (football, soccer, rugby). It does its job in a single file, with no framework dependencies beyond *requests* and *BeautifulSoup4*, and can be re-run any time to refresh the snapshot.

The design philosophy is deliberate: a small, dependable script that one person can read in five minutes and modify in ten. There is no headless browser, no ORM, no queue — just two HTTP passes (discovery on the collection page, then per-product extraction), a keyword filter, and a CSV writer. Images are downloaded as binary content with one extra request per kept row.

What lands on disk: **products.csv** capturing every product page the scraper discovered, **filtered_products.csv** with only the football / soccer / rugby items plus a description column, and an *images/* folder with one product thumbnail per kept row. The output is ready to feed into a buyer's spreadsheet, a price-tracker, or a downstream catalog system.

36

Products discovered

11

Sports items kept

11

Images downloaded

1

Python file

SECTION 02

Crawl Pipeline

Two HTTP passes, one keyword filter, two on-disk outputs.

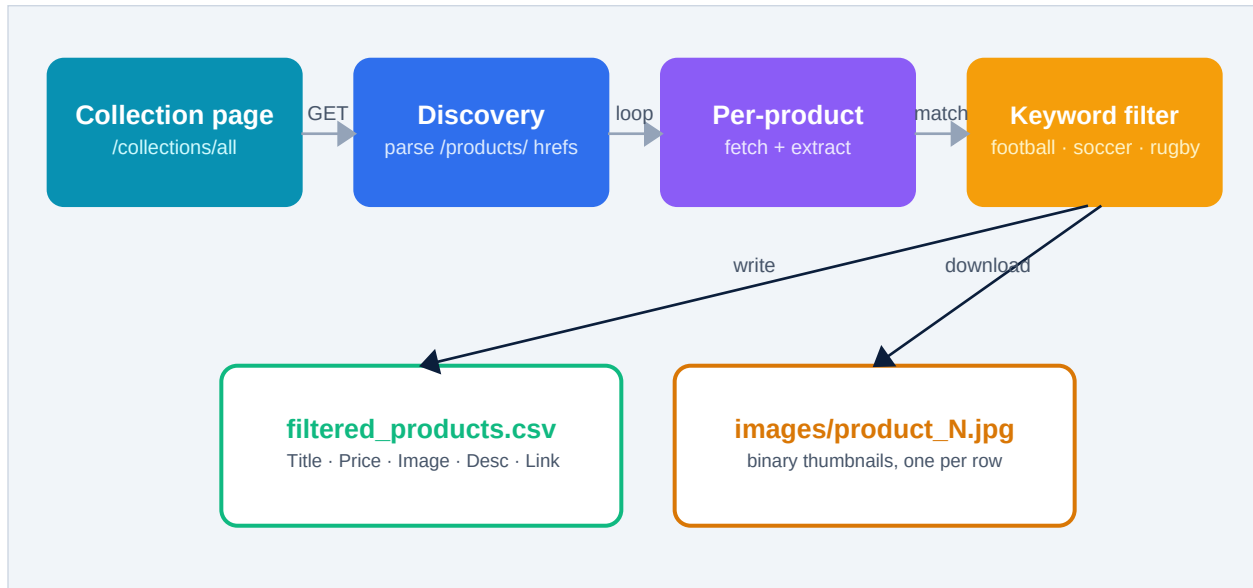


Figure — Collection page → discovery → per-product fetch → keyword filter → CSV + image folder.

Pass 1 — Discovery. The scraper fetches the collection page (*fcsuniforms.com/collections/all*) once, parses every anchor with *BeautifulSoup4*, and collects URLs that contain */products/*, de-duplicating as it goes. This is the only place the scraper has to know about site structure.

Pass 2 — Per-product extraction. Each discovered URL is fetched individually. From the product page the scraper pulls the *h1* title, the first \$-prefixed price span (while ignoring "art fee" spans), the first image inside the *.product* container, and the *.product__description* text. Image URLs that start with *//* are normalised to *https://*.

Output writeout. Rows that match the sport-keyword whitelist are appended to *filtered_products.csv* through Python's *csv.writer* in UTF-8 mode. The matching image is fetched in a separate binary request and written to *images/product_<i>N</i>.jpg*.

SECTION 03

Field Extraction Map

Exactly which HTML landmarks become which CSV columns.

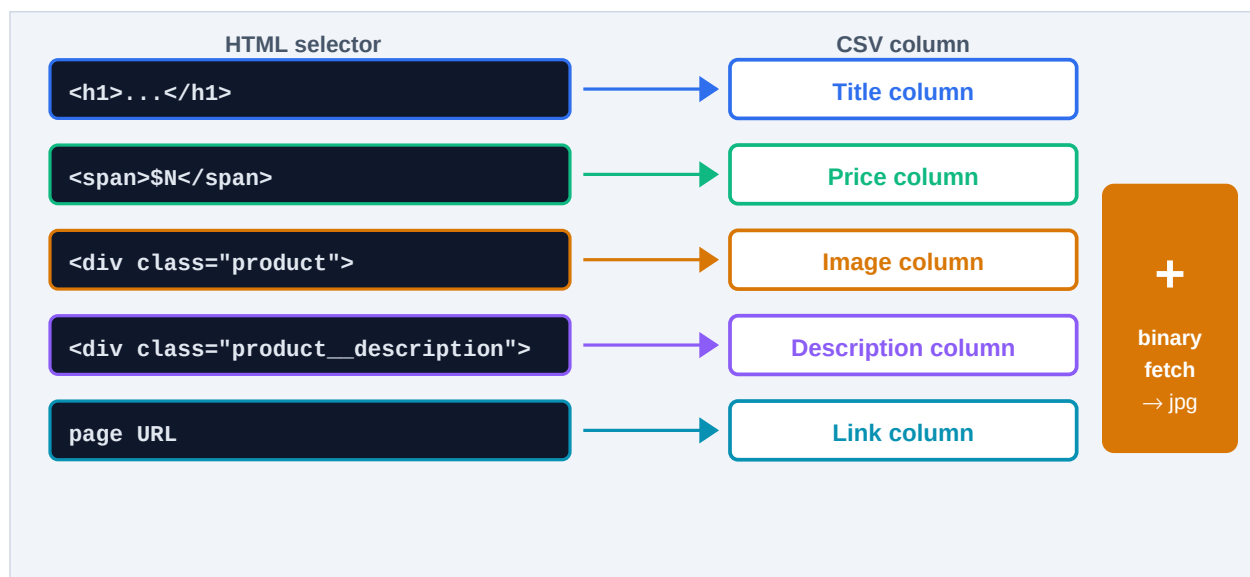


Figure — DOM landmarks on the left → CSV columns on the right; image URL also forks into a binary download.

The extraction layer is built around stable DOM landmarks rather than brittle CSS chains. The **h1** is the product title on every page. The **first \$-prefixed span** is the displayed price (with an exclusion for "art fee" spans, which would otherwise leak through). The **.product img** selector locks onto the main product image, and **.product__description** grabs the body copy. Each of those landmarks corresponds 1:1 to a CSV column.

The image URL takes two paths: it is written into the CSV as a string, and it is also handed to a binary `requests.get` that streams the bytes to disk as a `.jpg`. URLs starting with `//` get the `https:` scheme prepended so the binary fetch succeeds.

SECTION 04

Technology Stack

Three standard-library / pip-installable dependencies, no more.

Layer	Technology & Details
Language	Python 3 — single-file implementation, deterministic dependencies
HTTP	<i>requests</i> — collection page GET, per-product GET, and binary image GET
Parsing	<i>BeautifulSoup4</i> with the built-in <i>html.parser</i> — h1 title, \$-prefixed span price, .product img, .product__description
Filtering	Keyword whitelist (<i>football, soccer, rugby</i>) applied to product titles after lowercasing
CSV Output	Python's standard-library <i>csv.writer</i> with UTF-8 encoding for safe i18n quoting
Asset Storage	<i>os.makedirs</i> + binary file writes — one <i>images/product_<i>.jpg</i> per kept row
Runtime	No virtualenv, no .lock files, no Docker — pip install requests beautifulsoup4 and the script runs

Why this stack. The smaller the dependency tree, the longer a scraper survives. requests + BeautifulSoup is the most stable web-scraping pair in the Python ecosystem, and choosing the standard library's *html.parser* over *lxml* avoids a C-build dependency entirely. The script can be cloned and run on any machine with Python 3 installed and pip available, no configuration required.

SECTION 05

The Challenge

Product catalogs rarely have clean APIs and almost never let you bulk-download images.

A buyer evaluating a sportswear supplier needs a structured view of what the supplier carries — titles, prices, links, images — narrowed to the categories they actually care about. On almost every e-commerce platform, that view does not exist as an export. The catalog is paginated, the export button is buried in an admin-only area, and the images are served behind CDNs that obfuscate the original URLs.

The brief was a small, dependable scraper the user could re-run whenever the catalog changed. Specifically it had to: **(1)** discover every product page on a public storefront with one entry-point URL; **(2)** extract the title, price, image and description from each product without relying on a brittle CSS chain; **(3)** filter the catalog by sport keywords so the output stayed buyer-focused; and **(4)** download the matching images alongside the CSV, ready for inclusion in a procurement document or product database.

Two side-constraints made the implementation choices: it had to be a **single file** the user could read end-to-end, and it had to avoid any browser automation overhead. That ruled out Selenium, Playwright, and headless Chrome — and pushed the design toward *requests* + *BeautifulSoup4*.

SECTION 06

The Approach & Filter Funnel

Two-pass crawl, stable selectors, keyword whitelist, binary asset download.

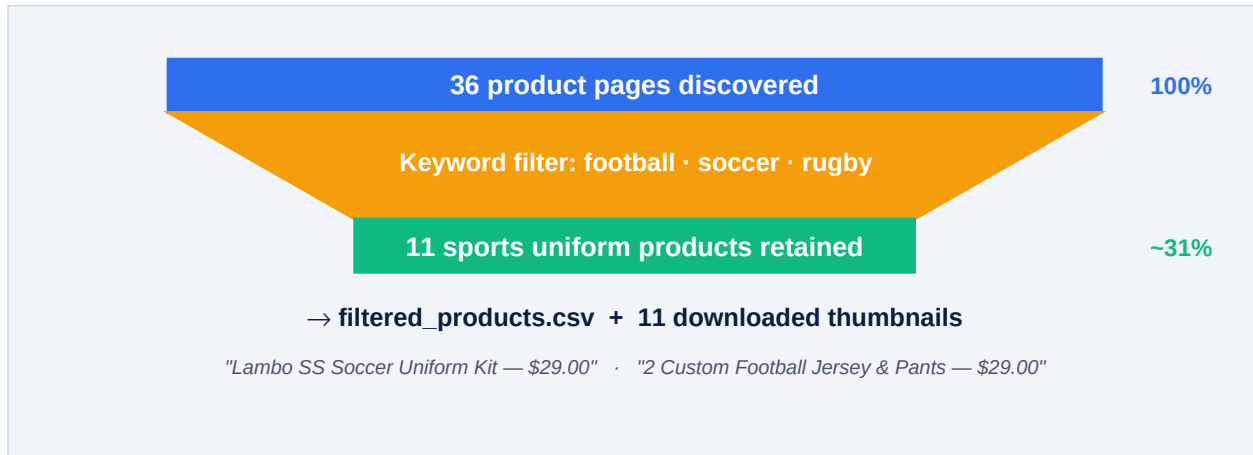


Figure — From 36 discovered products down to 11 retained — keyword filter does most of the narrowing.

The pipeline is a two-pass crawl. The first pass exists purely to enumerate the catalog: it hits the collection page once and harvests every `/products/` link. The second pass visits each product page and pulls structured fields. Decoupling discovery from extraction means the crawl can be paused, resumed, or paginated later without changing the extraction logic.

The keyword filter sits between extraction and writeout. Titles are lowercased and matched against a tiny whitelist (*football, soccer, rugby*). Products that match are written to the CSV and have their main image downloaded; products that don't are silently skipped. The funnel above shows the real numbers from the most recent run: 36 products discovered, 11 retained.

Image download is intentionally synchronous and per-row. There is no thread pool and no async queue. The price of that simplicity is a slower run; the benefit is zero failure modes from concurrent file writes, no rate-limit issues, and a script that fails politely if the network drops mid-run.

SECTION 07

Code Spotlight

Two short snippets — discovery + filter loop, and image-URL normalisation + binary write.

Two-pass discovery + keyword filter

scraper.py

```
# Pass 1 - discover all /products/ links from the collection page
response = requests.get(base_url + "/collections/all")
soup = BeautifulSoup(response.text, "html.parser")
product_links = []
for link in soup.find_all("a"):
    href = link.get("href")
    if href and "/products/" in href:
        full = base_url + href
        if full not in product_links:
            product_links.append(full)

# Pass 2 - visit each product, keyword-filter, write row
keywords = ["football", "soccer", "rugby"]
for i, url in enumerate(product_links):
    soup = BeautifulSoup(requests.get(url).text, "html.parser")
    title = soup.find("h1").text.strip()
    if not any(k in title.lower() for k in keywords):
        continue
```

Image URL normalisation + binary download

scraper.py

```
# Locate image inside .product container, normalise URL, download as JPG
image = ""
product_section = soup.find("div", { "class": "product" })
if product_section:
    img = product_section.find("img")
    if img: image = img.get("src")

# Protocol-relative URLs (//...) need the scheme prepended
if image.startswith("//"):
    image = "https:" + image

if image:
    img_data = requests.get(image).content
    file_name = f"images/product_{i}.jpg"
    with open(file_name, "wb") as f:
        f.write(img_data)
```

SECTION 08

Outcome & Engagement

A small, dependable, re-runnable view of a public catalog.

The scraper does what a one-off browser session would take an hour of manual copy-pasting to achieve, in one command. The user gets a clean CSV they can open in Excel, a folder of product thumbnails ready to drop into a procurement document, and a re-run path the moment the supplier updates the catalog.

The codebase is built to be extended in obvious directions: add another sport to the keyword list, paginate the collection crawl, or wrap the per-product fetch in a polite rate-limiter. None of those changes require restructuring the script — they fit inside the existing two-pass shape.

- › **Two-pass crawl** — discovery decoupled from extraction; the crawl can be paused or resumed without changing the extraction layer.
- › **Resilient selectors** — stable DOM landmarks (h1, .product img, .product__description) instead of fragile CSS chains.
- › **Keyword whitelist** — buyer-focused filter applied to lowercased titles; configurable in one line.
- › **Asset download** — main product image fetched as binary, normalised, and written per-row with sequential filenames.
- › **UTF-8 CSV** — fields written through Python's csv writer for safe quoting and i18n-clean product names.
- › **Zero infrastructure** — one file, two pip dependencies, no virtualenv or container required.

LET'S TALK

Mudasir Shafiq

Full-stack & data engineer

Email chmudasir.fr@gmail.com

Stacks Python • Airflow • BigQuery • Django • React • Docker